Ising model (ferromagnetism)

Simulation practice 2

Nima yadollahi – 40111046

Introduction

The Ising model is a mathematical framework used in statistical mechanics to describe phase transitions (**Fig1.1**), especially in ferromagnetic systems. Originally proposed by Ernst Ising in 1925, this model offers insights into how microscopic interactions can lead to macroscopic phenomena like magnetism. It consists of discrete variables representing the magnetic dipole moments of atomic "spins," which can be in one of two states: +1 or -1. These spins are arranged on a graph, typically a lattice, allowing each spin to interact with its neighboring spins.

Ernst Ising solved the one-dimensional Ising model in his 1924 thesis, demonstrating that it does not exhibit a phase transition. However, the two-dimensional square-lattice Ising model is more complex and was analytically solved much later by Lars Onsager in 1944. This model is usually approached using the transfer-matrix method, although there are alternative methods related to quantum field theory.

For dimensions greater than four, the phase transition in the Ising model is described by mean-field theory.

In this context, we will investigate the two-dimensional Ising model without an external magnetic field.

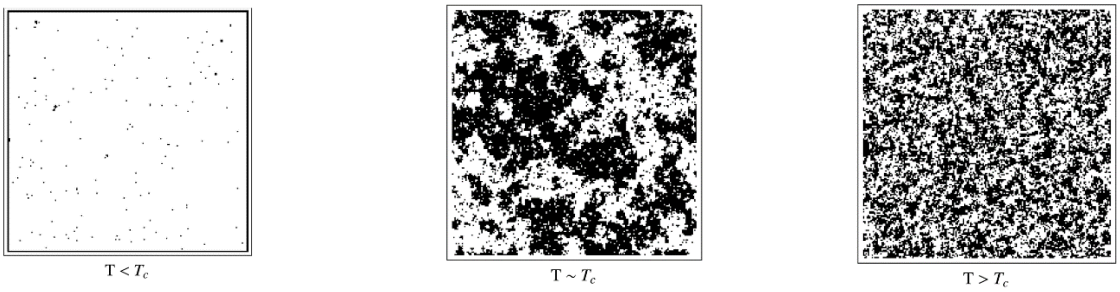


Fig 1.1 Phase Transition of the System

Abstract

The probability of a system in thermal equilibrium at a temperature bath and energy is as follows:

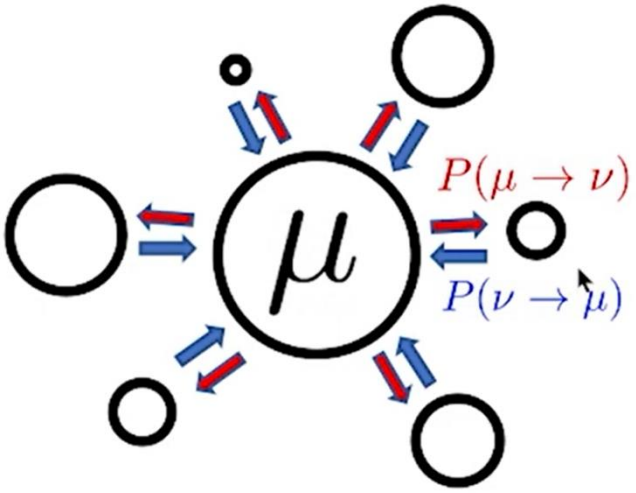
$$p_\mu = \frac{1}{z} e^{-\beta E_\mu}$$

Where Z is the partition function $z = \Sigma_\mu e^{-\beta E_\mu}$.

At equilibrium, the following conditions must be true:

$$\Sigma_\mu p_\mu P(\mu \to \nu) = \Sigma_\nu p_\nu P(\nu \to \mu)$$

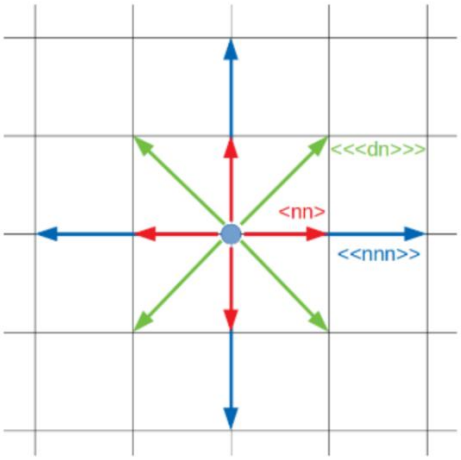
Where $P(\nu \to \mu)$ is the probability of going from state μ to ν .



The total energy E_μ in this exercise is:

$$E = -J_1 \sum_{<ij>} s_i s_j - J_2 \sum_{<<ij>>} s_i s_j - J_3 \sum_{<<<ij>>>} s_i s_j$$

In this context s_i represents the spin of a single particle in the lattice (-1 or +1) and the sum over <i,j>, <<i,j>> and <<<i,j>>> indicates that we are summing over first nearest neighbors, second nearest neighbors, and diagonal neighbors, respectively, for all points in the lattice.



$$\frac{J_1}{k} = 1 \, , \frac{J_2}{k} = 0.5 \, , \frac{J_3}{k} = 0.2$$

Now we can write:

$$\frac{P(\mu \to \nu)}{P(\nu \to \mu)} = \frac{p_\nu}{p_\mu} = e^{-\beta(E_\mu - E_\nu)}$$

Metropolis algorithm:

1. Call the current state μ.

2. Pick a random particle on the lattice and flip the spin sign. Call the state v. We want to find the probability that. $P(\mu \to \nu)$ that we'll accept this new state

3.

* If $E_\mu < E_v$ then set $P(v \to \mu) = 1$ and thus by the detailed balance equation $P(\mu \to \nu) = e^{-\beta(E_v - E_\mu)}$.

* if $E_\mu > E_v$ then set $P(\mu \to v) = 1$. Still satisfies detailed balance!

4. Change to state v (i.e. flip the spin of the particle) with the probabilities outlined above.

5. Go back to step 1. Repeat the whole thing many many times and eventually, you'll force out an equilibrium state.

Results & python code

Import Libraries & Define Constants

[3]:
```
import numpy as np
from numba import njit
import matplotlib.pyplot as plt
from PIL import Image
```

[48]:
```
k = 1.38 # Boltzman constant
J1 = 1.0 * k
J2 = 0.5 * k
J3 = 0.2 * k
```

[5]:
```
N = 20
steps = 1000
temperatures = [2.25, 4, 7]
```

I only used three temperatures, but the code runs well at other temperatures.

Functions

[7]:
```
def lattice(L):
    return np.random.choice(np.array([-1, 1]), size=(L, L))
```

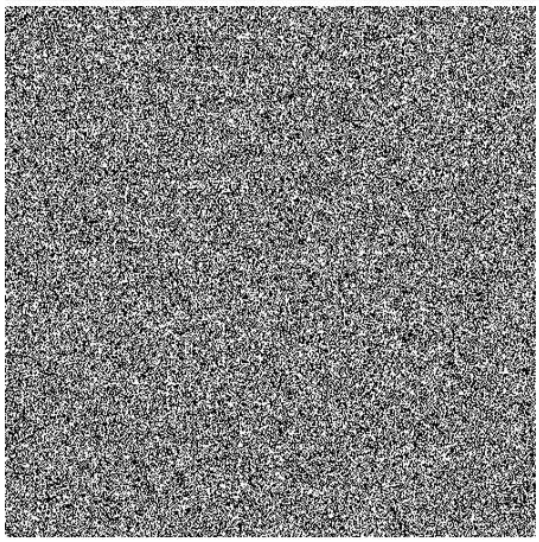
[8]:
```
lattice(100)
```

[8]:
```
array([[ 1,  1, -1, ..., -1,  1,  1],
       [-1, -1, -1, ..., -1,  1, -1],
       [ 1,  1,  1, ..., -1,  1, -1],
       ...,
       [-1, -1, -1, ...,  1,  1,  1],
       [-1,  1,  1, ...,  1,  1, -1],
       [ 1,  1,  1, ...,  1, -1,  1]])
```

[9]:
```
def image_this(field):
    return Image.fromarray(np.uint8((field + 1) * 0.5 * 255))
```

[10]:
```
image_this(lattice(500))
```

[10]:



[11]:
```
def compute_energy(field):
    L = field.shape[0]
    total_energy = 0
    for i in range(L):
        for j in range(L):
            S = field[i, j]
            total_energy -= J1 * S * (
                field[i, (j+1)%L] + field[i, (j-1)%L] +
                field[(i+1)%L, j] + field[(i-1)%L, j]
            )
            total_energy -= J2 * S * (
                field[(i+1)%L, (j+1)%L] + field[(i-1)%L, (j-1)%L] +
                field[(i+1)%L, (j-1)%L] + field[(i-1)%L, (j+1)%L]
            )
            total_energy -= J3 * S * (
                field[(i+2)%L, j] + field[i, (j+2)%L] +
                field[(i-2)%L, j] + field[i, (j-2)%L]
            )
    return total_energy / (L**2)
```

The function ‘compute energy’ calculates energy for a specific field and step.

[12]:
```python
def metropolis(field, T):
    L = field.shape[0]
    for _ in range(L**2):
        i, j = np.random.randint(0, L, size=2)
        S = field[i, j]
        dE = 2 * S * (
            J1 * (field[i, (j+1)%L] + field[i, (j-1)%L] +
                  field[(i+1)%L, j] + field[(i-1)%L, j]) +
            J2 * (field[(i+1)%L, (j+1)%L] + field[(i-1)%L, (j-1)%L] +
                  field[(i+1)%L, (j-1)%L] + field[(i-1)%L, (j+1)%L]) +
            J3 * (field[(i+2)%L, j] + field[i, (j+2)%L] +
                  field[(i-2)%L, j] + field[i, (j-2)%L])
        )
        if dE <= 0 or np.random.rand() < np.exp(-dE / T):
            field[i, j] *= -1
```

[13]:
```python
def simulate(N, T, steps):
    field = lattice(N)
    magnetization = []
    energy = []

    for step in range(steps+1):
        metropolis(field, T)
        mag = np.abs(np.sum(field)) / (N**2)
        en = compute_energy(field) / (N**2)
        magnetization.append(mag)
        energy.append(en)

    return np.array(energy), np.array(magnetization)
```

[62]:
```python
def heat_capacity(energies, T):
    return (np.std(energies)**2) / (k*(T**2))
```

$$C_V = \frac{\partial \langle E \rangle}{\partial t} = \frac{1}{N}\left(\frac{\langle E^2 \rangle - \langle E \rangle^2}{K_B^2 T^2}\right)$$

Where

Cv: heat capacity per spin.

T: temperature.

K_B: Boltzmann constant.

<E>: Average energy of the system.

$<E^2>$: Average of the square of the energy.

N: Total number of spins in the system, which we calculate in the ‘energies’ list in the ‘simulate’ function.

Results

[54]:
```
results = {}
for T in temperatures:
    energies, magnetizations = simulate(N, T, steps)
    results[T] = {
        "energies": energies,
        "magnetizations": magnetizations,
        "heat_capacity": heat_capacity(energies, T)
    }
```

[58]:
```
plt.figure(figsize=(15, 8))

for idx, T in enumerate(temperatures):
    plt.subplot(2, len(temperatures), idx + 1)
    plt.plot(results[T]["energies"], label=f"Energy T={T}" , c='b')
    plt.xlabel("Time")
    plt.ylabel(f"Energy {N}x{N}")
    plt.title(f"T = {T}")
    plt.legend()

    plt.subplot(2, len(temperatures), len(temperatures) + idx + 1)
    plt.plot(results[T]["magnetizations"], label=f"Magnetization T={T}", color="green")
    plt.ylim(-1,2)
    plt.xlabel("Time")
    plt.ylabel(f"Magnetization {N}x{N}")
    plt.legend()

plt.tight_layout()
plt.show()
```

In this section, I plot results next to each other for better comparison.

[345]:
```
heat_capacities = [results[T]["heat_capacity"] for T in temperatures]
plt.figure(figsize=(8, 6))
plt.plot(temperatures, heat_capacities, marker='o', label="Heat Capacity",c='r')
plt.xlabel("Temperature (T)")
plt.ylabel(f"Heat Capacity {N}x{N}")
plt.title("Heat Capacity vs Temperature")
plt.legend()
plt.grid(True)
plt.savefig(f"Cv for{N}x{N}(just J1).png",dpi=900)
plt.show()
```

Questions

A) Magnetization and total energy for the system with different temperatures. Obtain the time and ensemble averages. Examine the magnetization and energy in the calculation.

[126]:
```
print(f"\t All energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    all_e = results[T]['energies'].sum()
    all_mg = results[T]['magnetizations'].sum()
    print(f"The Energy in 'T={T}' is  {all_e}")
    print(f"The Magnetization in 'T={T}' is  {all_mg}\n")
```

```
     All energy & magnetization for 20x20 lattice

The Energy in 'T=2.25' is  -23.054887200000035
The Magnetization in 'T=2.25' is  943.155

The Energy in 'T=4' is  -21.67158900000005
The Magnetization in 'T=4' is  850.6949999999999

The Energy in 'T=7' is  -9.127720199999999
The Magnetization in 'T=7' is  238.095
```

If we sum all the results in the list, we give all the energy and magnetization.

[124]:
```
print(f"\t ensemble averaging energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    ensemble_mean_e = results[T]['energies'].mean()
    ensemble_mean_mg = results[T]['magnetizations'].mean()
    print(f"The average of Energy in 'T={T}' is  {ensemble_mean_e}")
    print(f"The average of Magnetization in 'T={T}' is  {ensemble_mean_mg}\n")
```

```
     ensemble averaging energy & magnetization for 20x20 lattice

The average of Energy in 'T=2.25' is  -0.02303185534465538
The average of Magnetization in 'T=2.25' is  0.9422127872127872

The average of Energy in 'T=4' is  -0.02164993906093911
The average of Magnetization in 'T=4' is  0.8498451548451548

The average of Energy in 'T=7' is  -0.009118601598401598
The average of Magnetization in 'T=7' is  0.23785714285714285
```

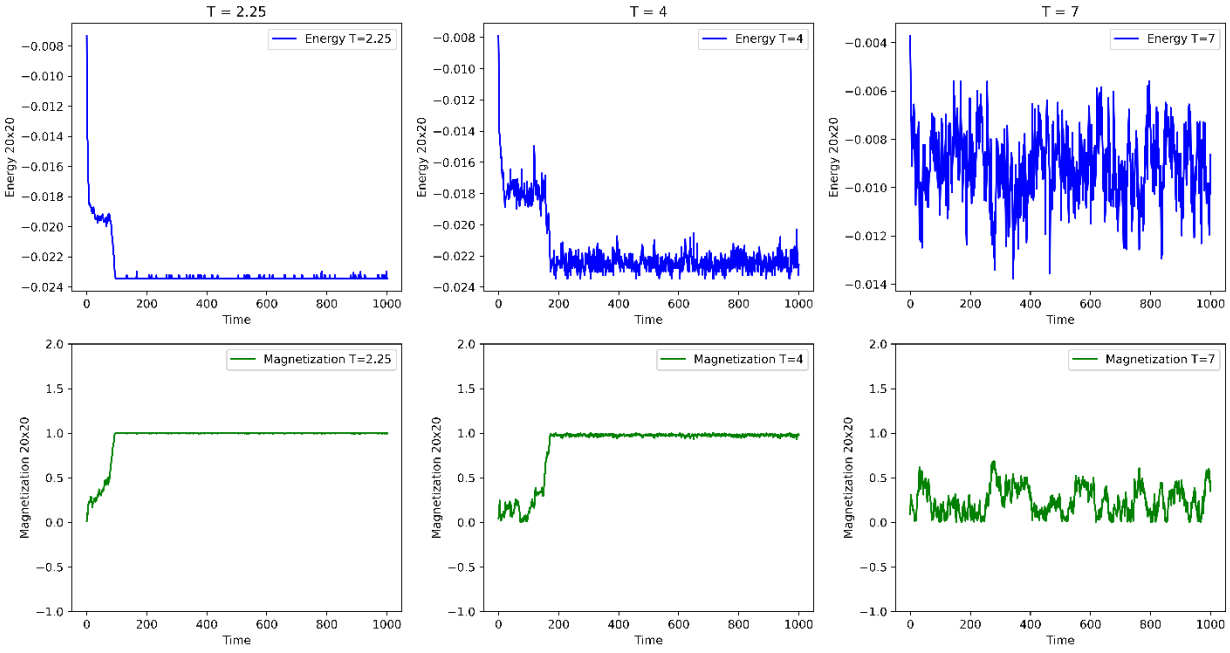
For ensemble averaging on energy and magnetization, I provide the mean in the results list returned by the ‘simulate’ function.

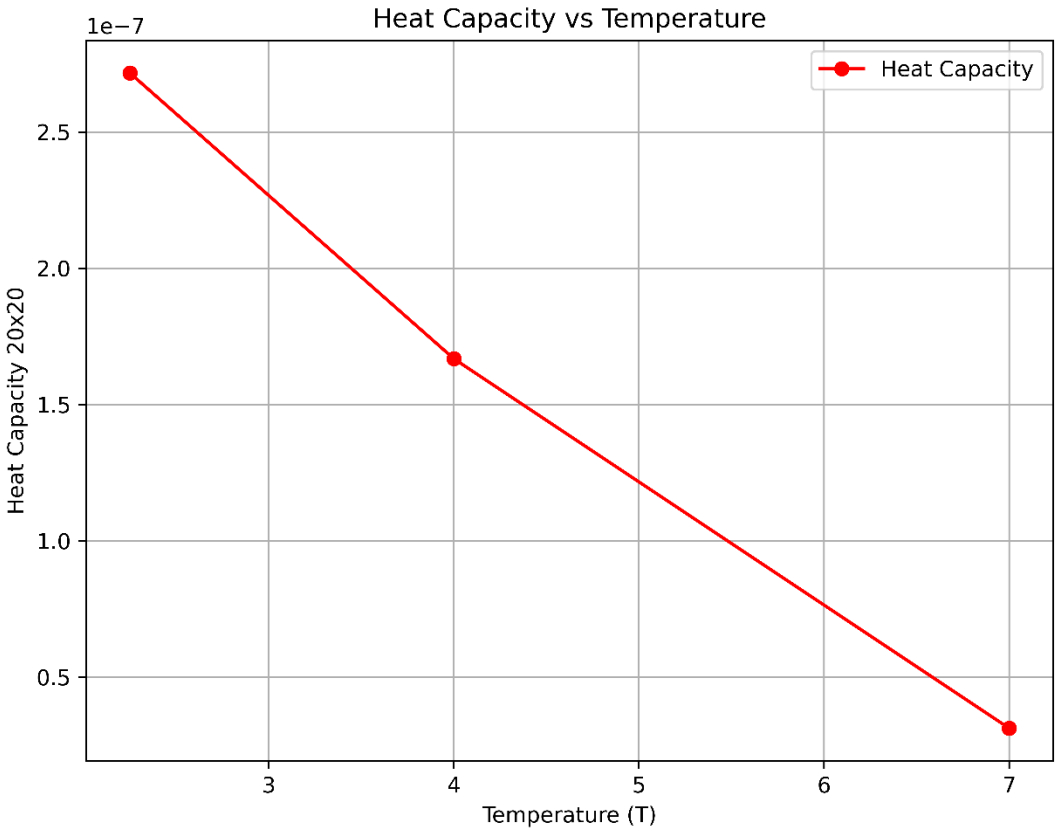
[122]:
```
print(f"\t time averaging energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    a , b = [] , []
    for i in range(20):
        a.append(results[T]['energies'][i])
        b.append(results[T]['magnetizations'][i])
    time_mean_e = sum(a)/len(a)
    time_mean_mg = sum(b)/len(b)
    print(f"The average of Energy in first 20 second for 'T={T}' is  {time_mean_e}")
    print(f"The mean Magnetization in first 20 second for 'T={T}' is  {time_mean_mg}\n")
```

```
        time averaging energy & magnetization for 20x20 lattice

The average of Energy in first 20 second for 'T=2.25' is  -0.017015400000000066
The mean Magnetization in first 20 second for 'T=2.25' is  0.19075000000000003

The average of Energy in first 20 second for 'T=4' is  -0.015312825000000049
The mean Magnetization in first 20 second for 'T=4' is  0.08975000000000001

The average of Energy in first 20 second for 'T=7' is  -0.007625879999999999
The mean Magnetization in first 20 second for 'T=7' is  0.14025
```

For time averaging on energy and magnetization, I provide the mean of the first 20 seconds (steps) in the results list returned by the ‘simulate’ function.

Plots for temperatures 2.25, 4, and 7 :





B) Determine the role of system size (number of spins in the system) in magnetization and energy.

Averaging results & plots for a 30x30 lattice:

[219]:
```
print(f"\t All energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    all_e = results[T]['energies'].sum()
    all_mg = results[T]['magnetizations'].sum()
    print(f"The Energy in 'T={T}' is  {all_e}")
    print(f"The Magnetization in 'T={T}' is  {all_mg}\n")
```

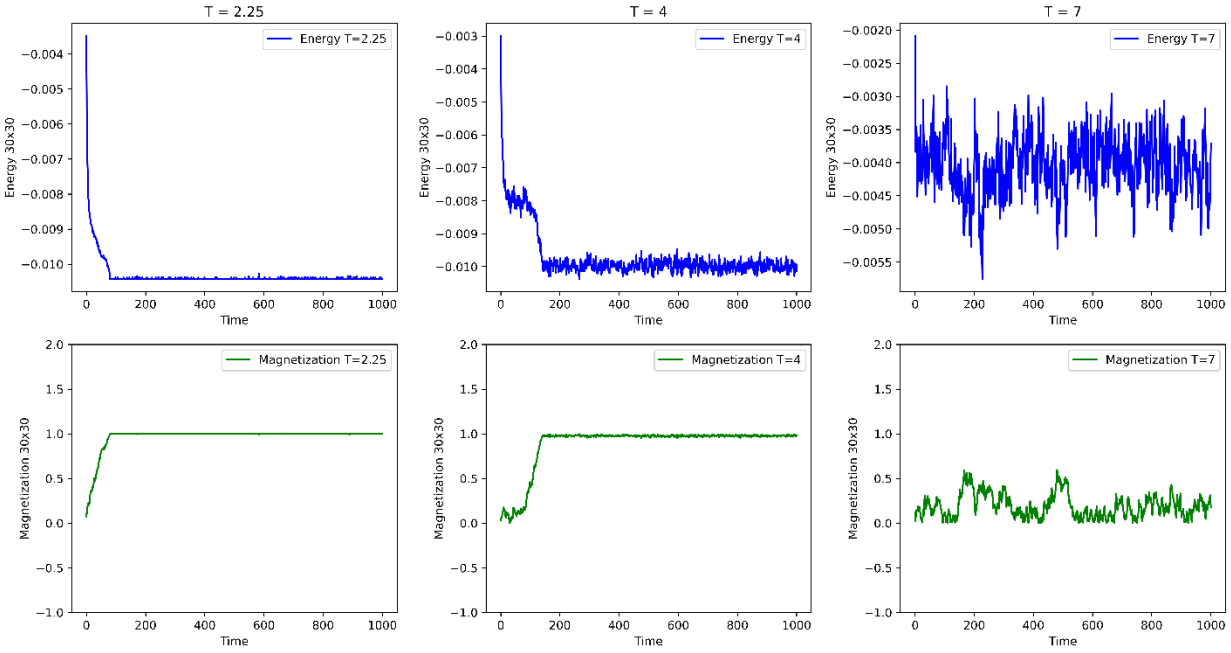
```
        All energy & magnetization for 30x30 lattice

The Energy in 'T=2.25' is  -10.331587733333828
The Magnetization in 'T=2.25' is  969.3644444444443

The Energy in 'T=4' is  -9.715442607407862
The Magnetization in 'T=4' is  883.6533333333333

The Energy in 'T=7' is  -4.046856740741195
The Magnetization in 'T=7' is  190.51999999999998
```

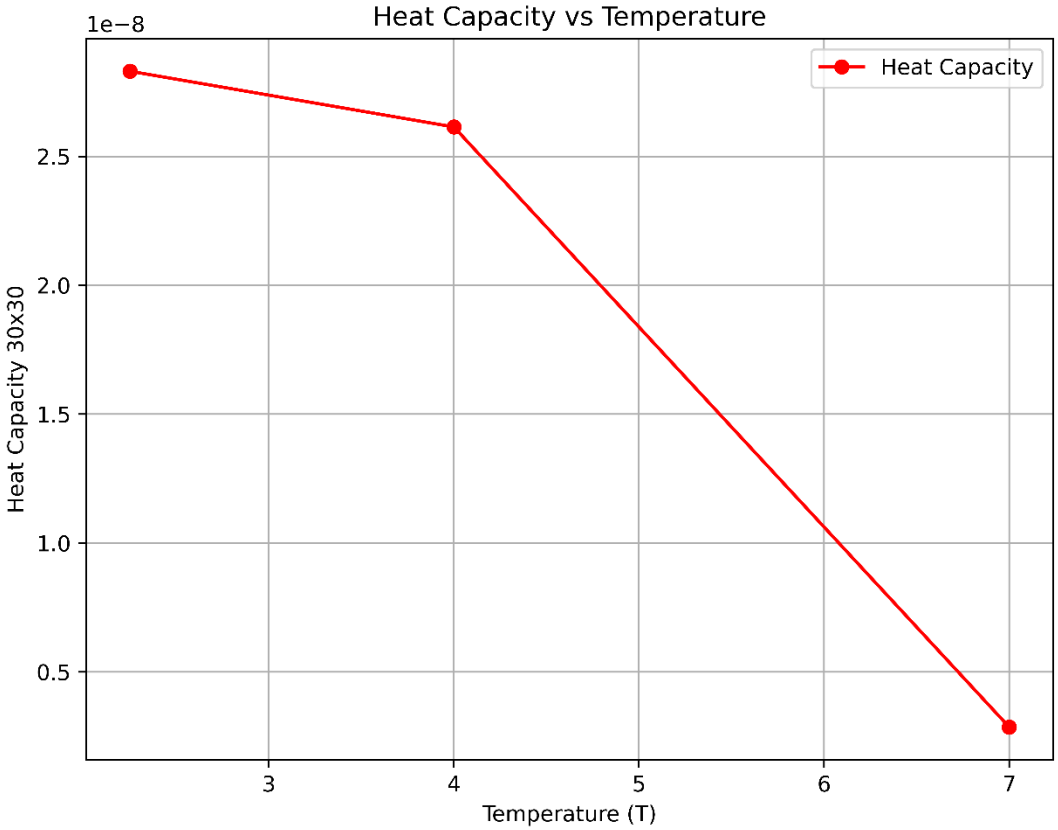
[220]:
```
print(f"\t ensemble averaging energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    ensemble_mean_e = results[T]['energies'].mean()
    ensemble_mean_mg = results[T]['magnetizations'].mean()
    print(f"The average of Energy in 'T={T}' is  {ensemble_mean_e}")
    print(f"The average of Magnetization in 'T={T}' is  {ensemble_mean_mg}\n")
```

```
        ensemble averaging energy & magnetization for 30x30 lattice

The average of Energy in 'T=2.25' is  -0.01032126646686696
The average of Magnetization in 'T=2.25' is  0.9683960483960482

The average of Energy in 'T=4' is  -0.009705736870537324
The average of Magnetization in 'T=4' is  0.8827705627705628

The average of Energy in 'T=7' is  -0.004042813660413706
The average of Magnetization in 'T=7' is  0.19032967032967033
```

[221]:
```
print(f"\t time averaging energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    a , b = [] , []
    for i in range(20):
        a.append(results[T]['energies'][i])
        b.append(results[T]['magnetizations'][i])
    time_mean_e = sum(a)/len(a)
    time_mean_mg = sum(b)/len(b)
    print(f"The average of Energy in first 20 second for 'T={T}' is  {time_mean_e}")
    print(f"The mean Magnetization in first 20 second for 'T={T}' is  {time_mean_mg}\n")
```

```
        time averaging energy & magnetization for 30x30 lattice

The average of Energy in first 20 second for 'T=2.25' is  -0.00789271407407435
The mean Magnetization in first 20 second for 'T=2.25' is  0.24955555555555559

The average of Energy in first 20 second for 'T=4' is  -0.006678450370370561
The mean Magnetization in first 20 second for 'T=4' is  0.10388888888888889

The average of Energy in first 20 second for 'T=7' is  -0.0037366311111111513
The mean Magnetization in first 20 second for 'T=7' is  0.12566666666666665
```





Averaging results & plots for a 50x50 lattice:

[143]:
```
print(f"\t All energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    all_e = results[T]['energies'].sum()
    all_mg = results[T]['magnetizations'].sum()
    print(f"The Energy in 'T={T}' is  {all_e}")
    print(f"The Magnetization in 'T={T}' is  {all_mg}\n")
```

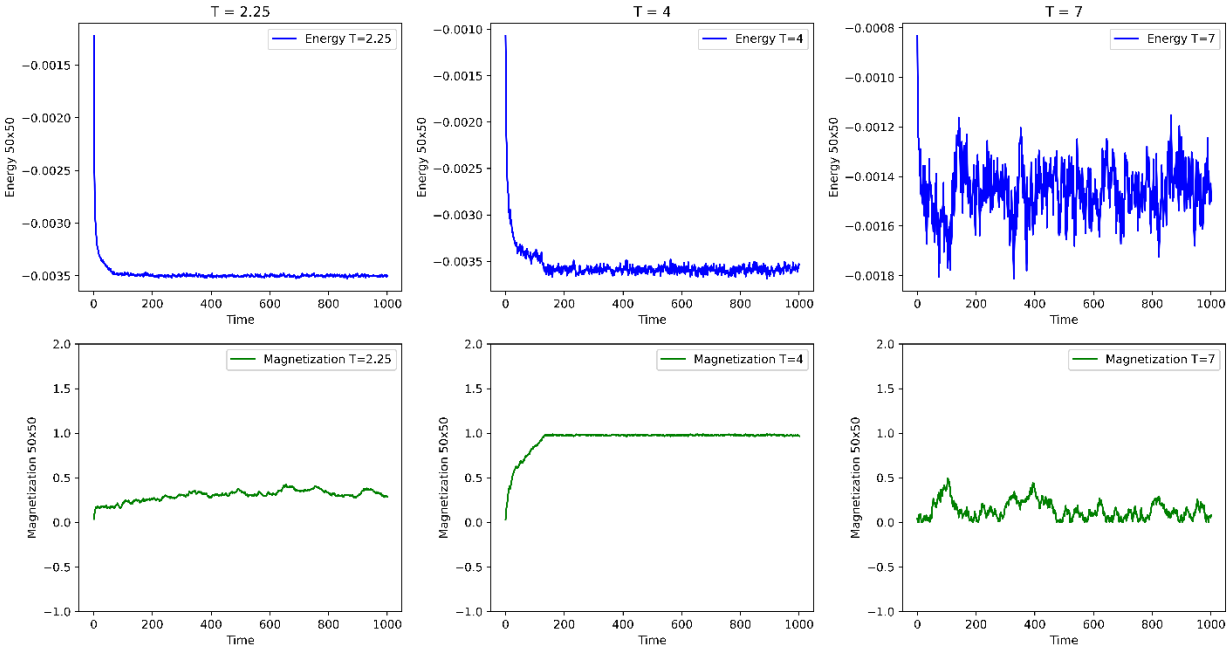
```
        All energy & magnetization for 50x50 lattice

The Energy in 'T=2.25' is  -3.485842552319893
The Magnetization in 'T=2.25' is  301.1696

The Energy in 'T=4' is  -3.5538093849598815
The Magnetization in 'T=4' is  939.3384

The Energy in 'T=7' is  -1.4611249228800502
The Magnetization in 'T=7' is  136.4272
```

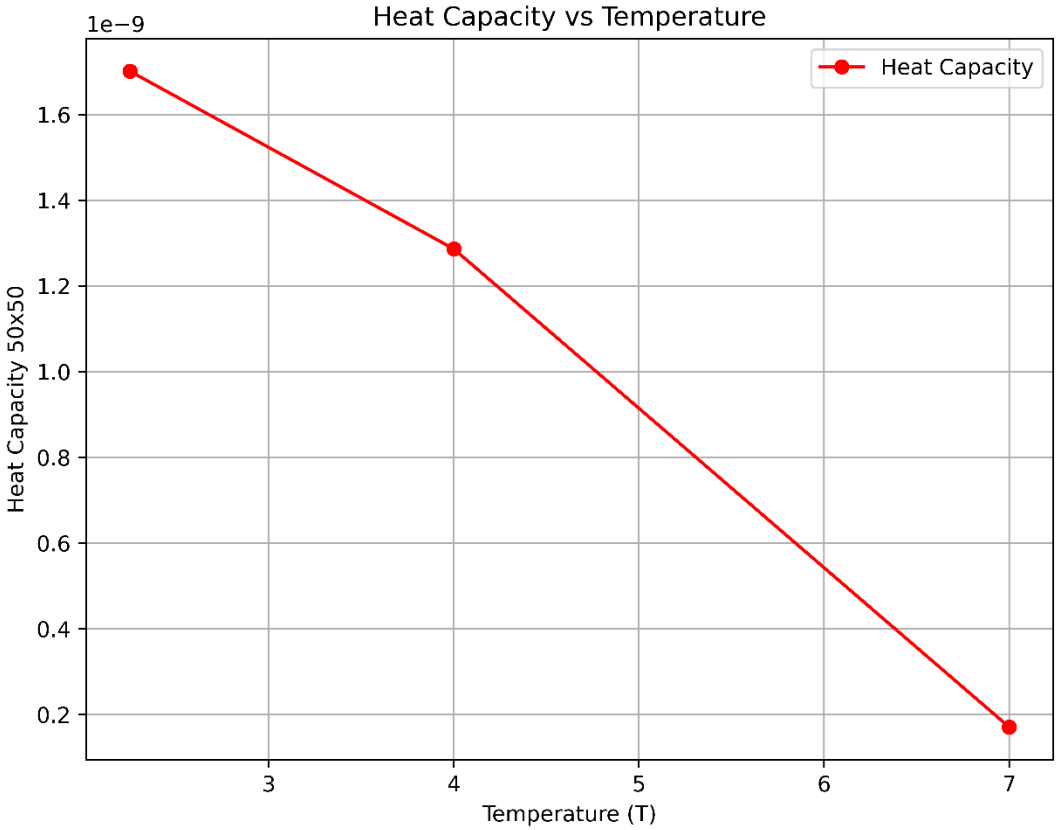
[144]:
```
print(f"\t ensemble averaging energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    ensemble_mean_e = results[T]['energies'].mean()
    ensemble_mean_mg = results[T]['magnetizations'].mean()
    print(f"The average of Energy in 'T={T}' is  {ensemble_mean_e}")
    print(f"The average of Magnetization in 'T={T}' is  {ensemble_mean_mg}\n")
```

```
        ensemble averaging energy & magnetization for 50x50 lattice

The average of Energy in 'T=2.25' is  -0.003482360192127765
The average of Magnetization in 'T=2.25' is  0.30086873126873126

The average of Energy in 'T=4' is  -0.0035502591258340477
The average of Magnetization in 'T=4' is  0.9384

The average of Energy in 'T=7' is  -0.001459665257622428
The average of Magnetization in 'T=7' is  0.1362909090909091
```

[145]:
```
print(f"\t time averaging energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    a , b = [] , []
    for i in range(20):
        a.append(results[T]['energies'][i])
        b.append(results[T]['magnetizations'][i])
    time_mean_e = sum(a)/len(a)
    time_mean_mg = sum(b)/len(b)
    print(f"The average of Energy in first 20 second for 'T={T}' is  {time_mean_e}")
    print(f"The mean Magnetization in first 20 second for 'T={T}' is  {time_mean_mg}\n")
```

```
        time averaging energy & magnetization for 50x50 lattice

The average of Energy in first 20 second for 'T=2.25' is  -0.002940146303999973
The mean Magnetization in first 20 second for 'T=2.25' is  0.14492

The average of Energy in first 20 second for 'T=4' is  -0.0026283413760000104
The mean Magnetization in first 20 second for 'T=4' is  0.28956

The average of Energy in first 20 second for 'T=7' is  -0.0013270256640000497
The mean Magnetization in first 20 second for 'T=7' is  0.03928
```





Averaging results & plots for a 100x100 lattice:

[181]:
```
print(f"\t All energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    all_e = results[T]['energies'].sum()
    all_mg = results[T]['magnetizations'].sum()
    print(f"The Energy in 'T={T}' is  {all_e}")
    print(f"The Magnetization in 'T={T}' is  {all_mg}\n")
```

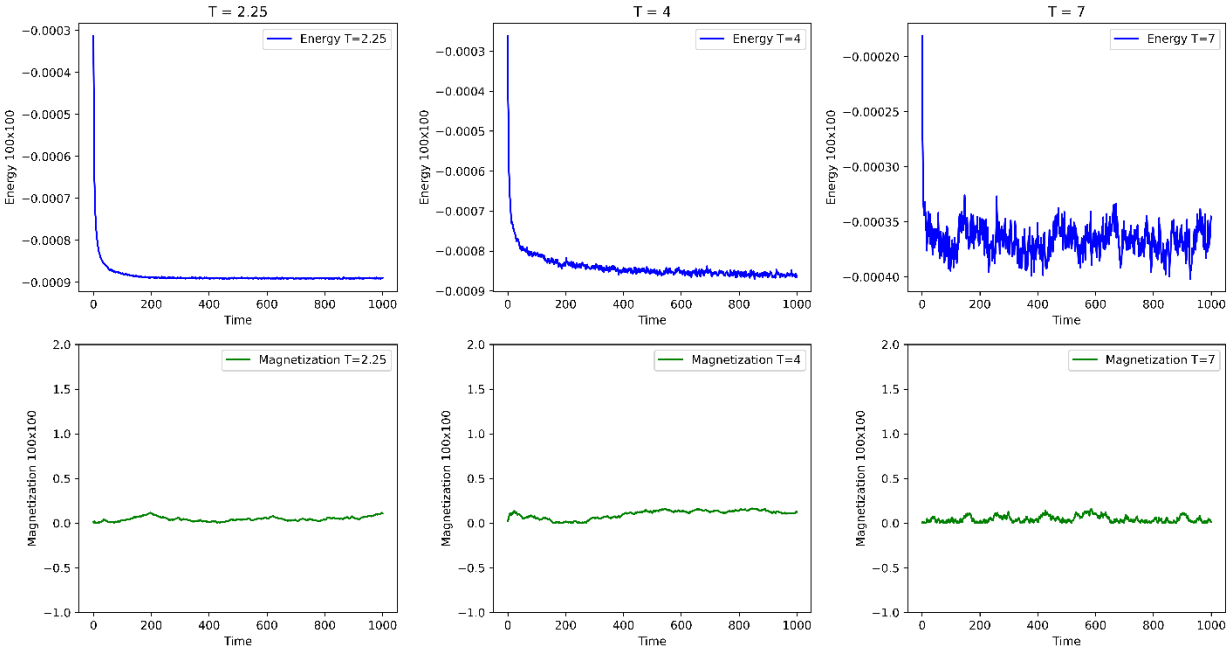
```
        All energy & magnetization for 100x100 lattice

The Energy in 'T=2.25' is  -0.8851365926400379
The Magnetization in 'T=2.25' is  45.7584

The Energy in 'T=4' is  -0.8405327625600094
The Magnetization in 'T=4' is  99.087

The Energy in 'T=7' is  -0.36653546303997425
The Magnetization in 'T=7' is  44.370999999999995
```

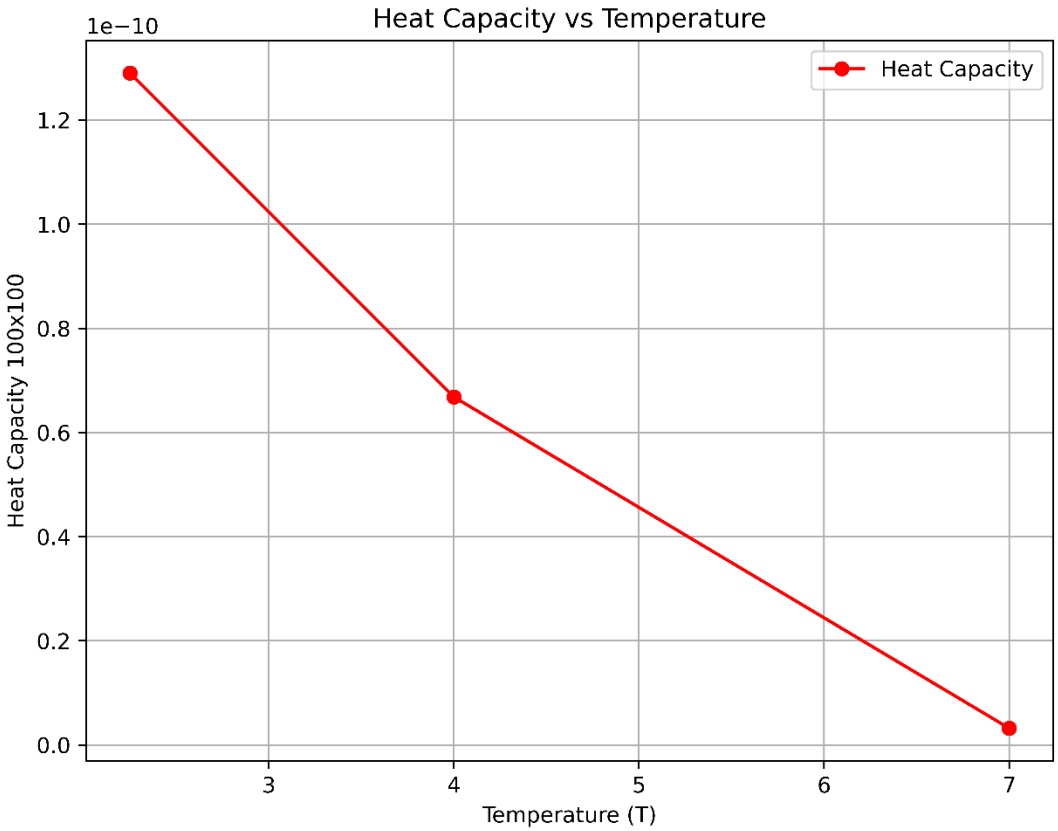
[182]:
```
print(f"\t ensemble averaging energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    ensemble_mean_e = results[T]['energies'].mean()
    ensemble_mean_mg = results[T]['magnetizations'].mean()
    print(f"The average of Energy in 'T={T}' is  {ensemble_mean_e}")
    print(f"The average of Magnetization in 'T={T}' is  {ensemble_mean_mg}\n")
```

```
      ensemble averaging energy & magnetization for 100x100 lattice

The average of Energy in 'T=2.25' is  -0.0008842523402997381
The average of Magnetization in 'T=2.25' is  0.045712687312687315

The average of Energy in 'T=4' is  -0.0008396930694905189
The average of Magnetization in 'T=4' is  0.09898801198801199

The average of Energy in 'T=7' is  -0.00036616929374622804
The average of Magnetization in 'T=7' is  0.044326673326673324
```

[183]:
```
print(f"\t time averaging energy & magnetization for {N}x{N} lattice\n")
for T in temperatures:
    a , b = [] , []
    for i in range(20):
        a.append(results[T]['energies'][i])
        b.append(results[T]['magnetizations'][i])
    time_mean_e = sum(a)/len(a)
    time_mean_mg = sum(b)/len(b)
    print(f"The average of Energy in first 20 second for 'T={T}' is  {time_mean_e}")
    print(f"The mean Magnetization in first 20 second for 'T={T}' is  {time_mean_mg}\n")
```

```
      time averaging energy & magnetization for 100x100 lattice

The average of Energy in first 20 second for 'T=2.25' is  -0.0007256205599999864
The mean Magnetization in first 20 second for 'T=2.25' is  0.00616

The average of Energy in first 20 second for 'T=4' is  -0.000644706743999964
The mean Magnetization in first 20 second for 'T=4' is  0.08789

The average of Energy in first 20 second for 'T=7' is  -0.00033008937599997735
The mean Magnetization in first 20 second for 'T=7' is  0.01216
```



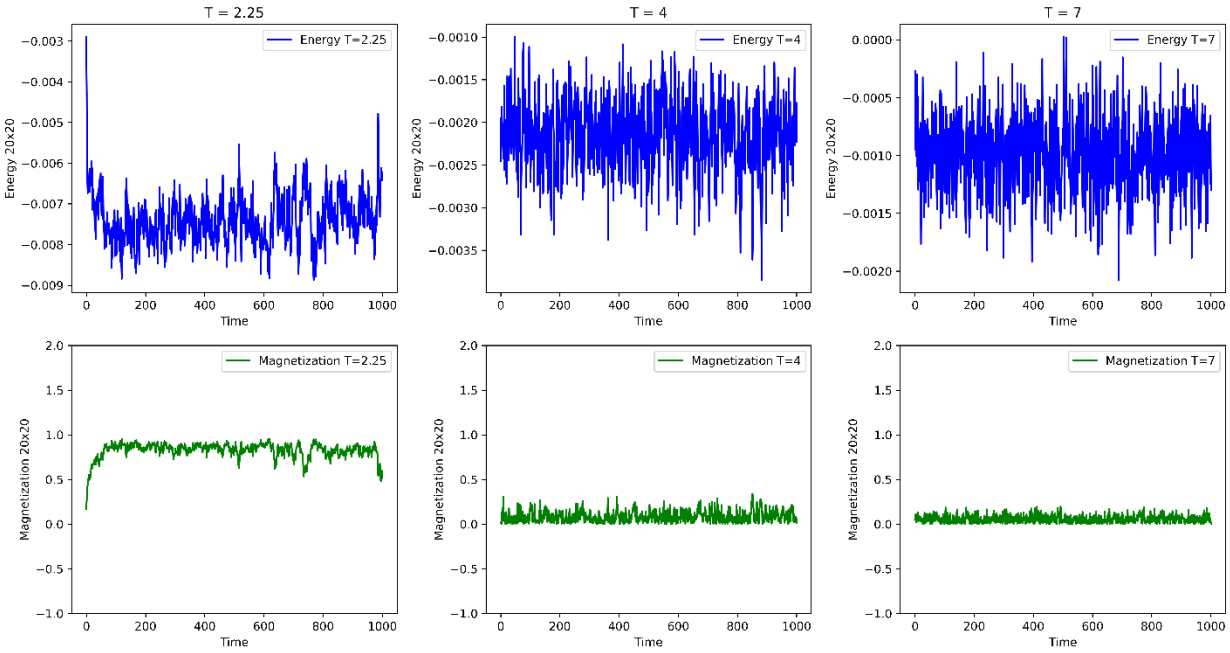


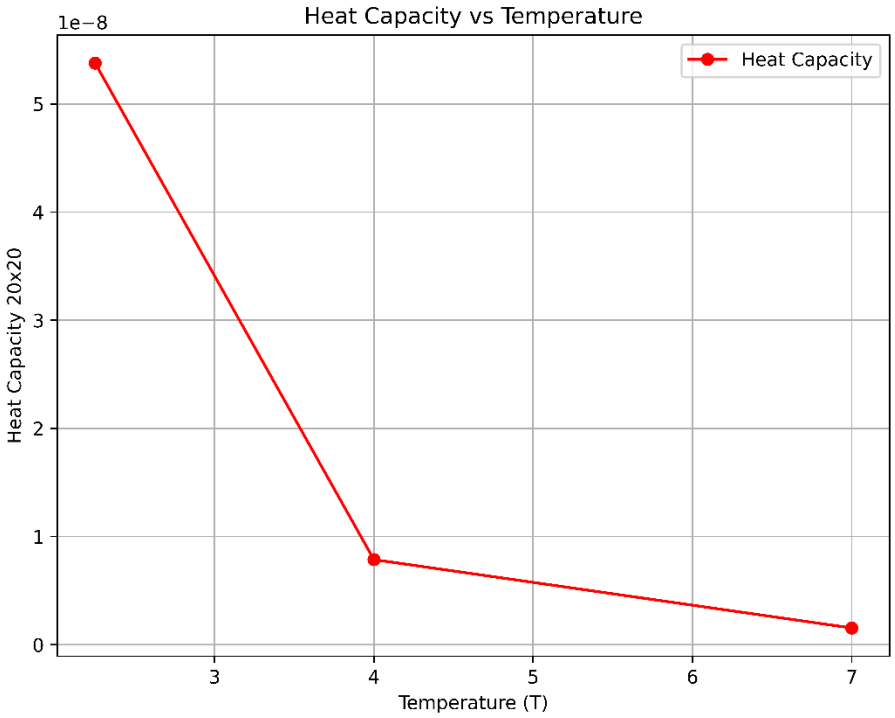
D) Calculate the role of second and third neighbor interactions in this simulation by changing the interaction constants.

Results & plots without J1:

[284]:
```
k = 1.38 # Boltzman constant
J1 = 0 * k
J2 = 0.5 * k
J3 = 0.2 * k
```

[245]:
```
N = 20
steps = 1000
temperatures = [2.25, 4, 7]
```

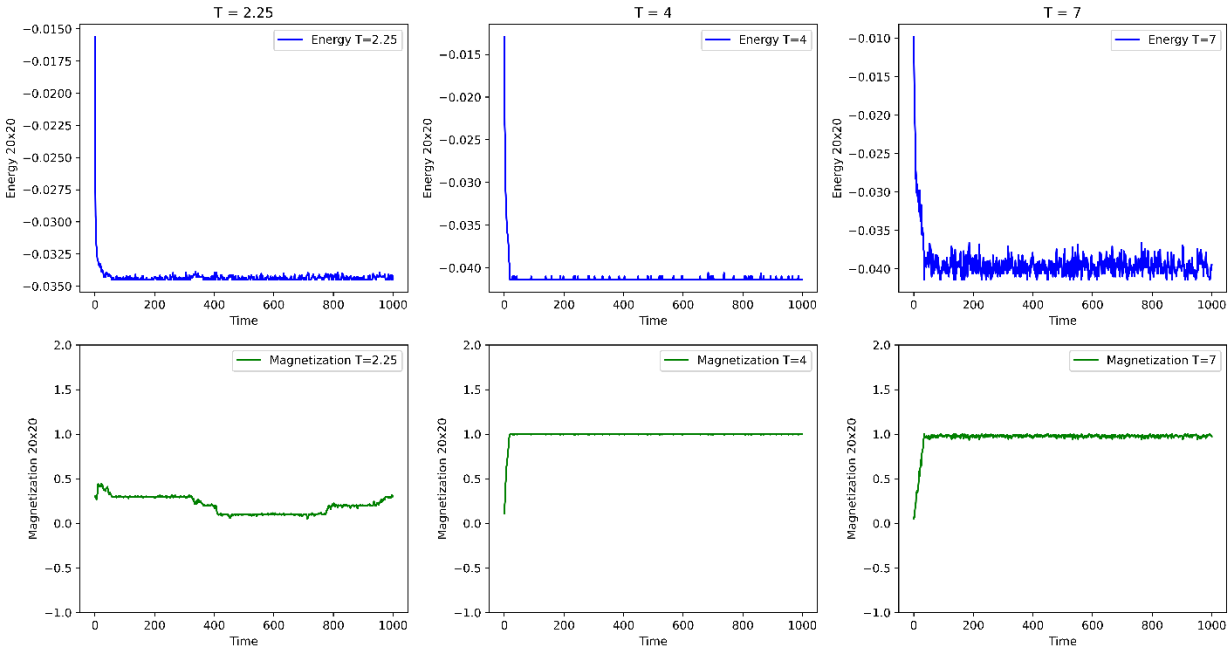

T = 2.25
Energy T=2.25
Energy 20x20
Time
T = 4
Energy T=4
Energy 20x20
Time
T = 7
Energy T=7
Energy 20x20
Time
Magnetization T=2.25
Magnetization 20x20
Time
Magnetization T=4
Magnetization 20x20
Time
Magnetization T=7
Magnetization 20x20
Time

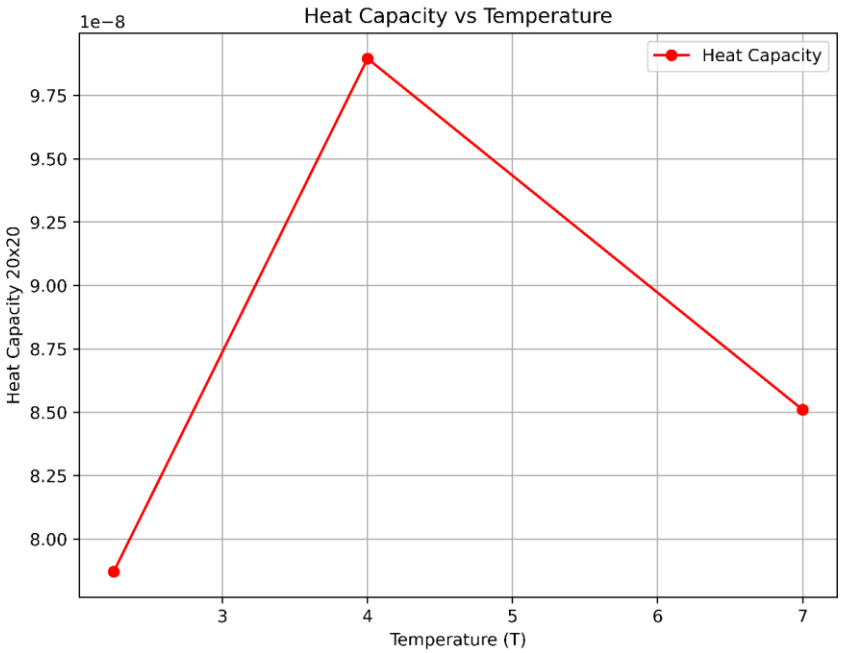

Heat Capacity vs Temperature
1e−8
Heat Capacity
Heat Capacity 20x20
Temperature (T)

Results with same J:

[244]:
```
k = 1.38 # Boltzman constant
J1 = 1 * k
J2 = 1 * k
J3 = 1 * k
```

[245]:
```
N = 20
steps = 1000
temperatures = [2.25, 4, 7]
```





Results with only J1

[324]:
```
k = 1.38 # Boltzman constant
J1 = 1 * k
J2 = 0 * k
J3 = 0 * k
```

[289]:
```
N = 20
steps = 1000
temperatures = [2.25, 4, 7]
```